

Application Properties

```
spring.application.name=testLeaflet
spring.datasource.url=jdbc:mysql://localhost:3306/test_leaflet
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

AuthController

```
package com.arnav.testLeaflet.Controller;

import com.arnav.testLeaflet.Service.AuthService;
import com.arnav.testLeaflet.model.Users;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/auth")
@CrossOrigin(origins = "http://localhost:3000")
public class AuthController {

    @Autowired
    private AuthService service;

    @GetMapping("/users")
    public List<Users> getUsers(){
        return service.getAllUsers();
    }

    @PostMapping("/user/register")
    public ResponseEntity<Users> addUser(@RequestBody Users user){
        Users savedUser = service.addUser(user);
        return ResponseEntity.status(HttpStatus.CREATED).body(savedUser);
    }
}
```

```

@PostMapping("/user/login")
public ResponseEntity<?> login(@RequestBody Users loginRequest){
    Users user = service.validateUser(
        loginRequest.getUsername(),
        loginRequest.getPassword()
    );
    if(user != null){
        return ResponseEntity.ok(user);
    }else{
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid Username or
password");
    }
}
}

```

AuthService

```

package com.arnav.testLeaflet.Service;

import com.arnav.testLeaflet.Repository.UserRepo;
import com.arnav.testLeaflet.model.Users;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class AuthService {

    @Autowired
    private UserRepo repo;

    public Users addUser(Users user) {
        return repo.save(user);
    }

    public Users validateUser(String username, String password) {
        Users user = repo.findByUsername(username).orElse(null);

        if (user != null && user.getPassword().equals(password)) {
            return user;
        }
    }
}

```

```

    }
    return null;
}

public List<Users> getAllUsers() {
    return repo.findAll();
}
}

```

AuthRepo

```

package com.arnav.testLeaflet.Repository;

import com.arnav.testLeaflet.model.Users;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface UserRepo extends JpaRepository<Users, Long> {
    Optional<Users> findByUsername(String username);
}

```

Users

```

package com.arnav.testLeaflet.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "users") // table name in DB
public class Users {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // auto-increment
    private Long id;

    @Column(length = 50, nullable = false, unique = true)

```

```
private String username;
```

```
@Column(length = 100, nullable = false, unique = true)
```

```
private String email;
```

```
@Column(length = 255, nullable = false)
```

```
private String password;
```

```
@Column(name = "created_at", nullable = false, updatable = false)
```

```
private LocalDateTime createdAt;
```

```
// Default constructor
```

```
public Users() {
```

```
    this.createdAt = LocalDateTime.now(); // sets default timestamp
```

```
}
```

```
// Getters and setters
```

```
public Long getId() {
```

```
    return id;
```

```
}
```

```
public void setId(Long id) {
```

```
    this.id = id;
```

```
}
```

```
public String getUsername() {
```

```
    return username;
```

```
}
```

```
public void setUsername(String username) {
```

```
    this.username = username;
```

```
}
```

```
public String getEmail() {
```

```
    return email;
```

```
}
```

```
public void setEmail(String email) {
```

```
    this.email = email;
```

```
}
```

```
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
  
public LocalDateTime getCreatedAt() {  
    return createdAt;  
}  
  
// No setter for createdAt to prevent manual changes  
}
```

AdminController

```
package com.arnav.testLeaflet.Controller;

import com.arnav.testLeaflet.Service.AdminService;
import com.arnav.testLeaflet.model.Admin;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@CrossOrigin(origins = "http://localhost:3000")
@RequestMapping("/auth")
public class AdminController {

    @Autowired
    private AdminService service;

    @PostMapping("/admin/login")
    public ResponseEntity<?> loginAdmin(@RequestBody Admin loginAdmin){
        Admin admin = service.LoginAsAdmin(loginAdmin.getUsername(),
loginAdmin.getPassword());
        if(admin != null){
            return ResponseEntity.ok(admin);
        } else {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid Username
and Password");
        }
    }
}
```

AdminService

```
package com.arnav.testLeaflet.Service;

import com.arnav.testLeaflet.Repository.AdminRepo;
import com.arnav.testLeaflet.model.Admin;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
```

```

@Service
public class AdminService {

    @Autowired
    private AdminRepo repo;

    public Admin LoginAsAdmin(String username, String password) {
        return repo.findByUsernameAndPassword(username, password)
            .orElseThrow(() -> new RuntimeException("Invalid username or password"));
    }
}

```

AdminRepo

```

package com.arnav.testLeaflet.Repository;

import com.arnav.testLeaflet.model.Admin;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface AdminRepo extends JpaRepository<Admin,Integer> {

    Optional<Admin> findByUsernameAndPassword(String username, String password);
}

```

Admin

```

package com.arnav.testLeaflet.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "admins") // table name in DB
public class Admin {

```

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY) // auto-increment
private Long id;

@Column(length = 50, nullable = false, unique = true)
private String username;

@Column(length = 100, nullable = false, unique = true)
private String email;

@Column(length = 255, nullable = false)
private String password;

// Getters and setters
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getPassword() {
    return password;
}
```

```

    }

    public void setPassword(String password) {
        this.password = password;
    }

    // No setter for createdAt to prevent manual changes
}

```

LocationController

```

package com.arnav.testLeaflet.Controller;

import com.arnav.testLeaflet.Service.LocationService;
import com.arnav.testLeaflet.model.Location;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@CrossOrigin(origins = "http://localhost:3000")
@RequestMapping("/location")
public class LocationController {

    @Autowired
    private LocationService service;

    @PostMapping("/setMarker")
    public ResponseEntity<Location> setLocationMarker(@RequestBody Location marker){
        Location saved = service.addMarker(marker);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @GetMapping("/getMarkers")
    public ResponseEntity<List<Location>> getMarkers(){
        List<Location> locate = service.getMarkerLocations();
    }
}

```

```

        if(locate.isEmpty()){
            return ResponseEntity.noContent().build();
        }else{
            return ResponseEntity.ok(locate);
        }
    }
}

@DeleteMapping("/deleteMarker/{id}")
public ResponseEntity<String> deleteMarker(@PathVariable Long id){
    try{
        service.deleteMarker(id);
        return ResponseEntity.ok("Marker Deleted Sucessfully");
    }catch(RuntimeException e){
        return ResponseEntity.badRequest().body(e.getMessage());
    }
}

@PutMapping("/verifyMarker/{id}")
public ResponseEntity<String> verifyMarker(@PathVariable Long id){
    try{
        service.verifyMarker(id);
        return ResponseEntity.ok("Marker Verified");
    }catch(RuntimeException e){
        return ResponseEntity.badRequest().body("Unable to verify");
    }
}
}

```

LocationService

```

package com.arnav.testLeaflet.Service;

import com.arnav.testLeaflet.Repository.LocationRepo;
import com.arnav.testLeaflet.model.Location;
import com.arnav.testLeaflet.model.Users;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

```

```

import java.util.List;

@Service
public class LocationService {

    @Autowired
    private LocationRepo repo;

    @PersistenceContext
    private EntityManager entityManager;

    @Transactional
    public Location addMarker(Location marker) {
        if (marker.getCreatedBy() == null || marker.getCreatedBy().getId() == null) {
            throw new RuntimeException("User ID is required for createdBy");
        }

        // Get managed reference to avoid transient entity error
        Users userRef = entityManager.getReference(Users.class,
marker.getCreatedBy().getId());

        marker.setCreatedBy(userRef);

        return repo.save(marker);
    }

    public List<Location> getMarkerLocations() {
        return repo.findAll();
    }

    public void deleteMarker(Long id) {
        if(repo.existsById(id)){
            repo.deleteById(id);
        }else{
            throw new RuntimeException("Not found with "+id);
        }
    }

    public void verifyMarker(Long id) {
        Location marker = repo.findById(id)

```

```

        .orElseThrow(() -> new RuntimeException("Marker not found with id: " + id));

        marker.setStatus("VERIFIED"); // update status
        repo.save(marker);
    }
}

```

LocationRepo

```

package com.arnav.testLeaflet.Repository;

import com.arnav.testLeaflet.model.Location;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface LocationRepo extends JpaRepository<Location,Long> {
}

```

Location

```

package com.arnav.testLeaflet.model;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;

@Entity
@Table(name = "locations")
public class Location {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private Double latitude;

    @Column(nullable = false)
    private Double longitude;

    @Column(length = 20, nullable = false)

```

```

private String status;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "created_by", nullable = false)
@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"}) // prevents JSON errors
private Users createdBy;

// Getters and setters
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }

public Double getLatitude() { return latitude; }
public void setLatitude(Double latitude) { this.latitude = latitude; }

public Double getLongitude() { return longitude; }
public void setLongitude(Double longitude) { this.longitude = longitude; }

public String getStatus() { return status; }
public void setStatus(String status) { this.status = status; }

public Users getCreatedBy() { return createdBy; }
public void setCreatedBy(Users createdBy) { this.createdBy = createdBy; }
}

```

CorsConfig

```

package com.arnav.testLeaflet;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {

```

```

@Override
public void addCorsMappings(CorsRegistry registry) {
    registry.addMapping("/**") // allow all endpoints
        .allowedOrigins("http://localhost:3000")
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders("*");
}
};
}
}

```

GlobalExceptionHandler

```
package com.example.demo.exception;
```

```

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;

```

```

import java.util.HashMap;
import java.util.Map;

```

```
@ControllerAdvice
```

```
public class GlobalExceptionHandler {
```

```
    // Handle specific exception
```

```
    @ExceptionHandler(ResourceNotFoundException.class)
```

```
    public ResponseEntity<Map<String, String>>
    handleResourceNotFound(ResourceNotFoundException ex) {
```

```
        Map<String, String> error = new HashMap<>();
```

```
        error.put("errorMessage", ex.getMessage());
```

```

        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    // Handle validation errors
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>>
    handleValidationExceptions(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage()));
        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
    }

    // Handle all other exceptions
    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, String>> handleGlobalException(Exception ex) {
        Map<String, String> error = new HashMap<>();
        error.put("errorMessage", ex.getMessage());
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}

```